

# Módulo 1: Introdução e Cultura Maker

## 1.1 O que são Sistemas Embarcados?

Um sistema embarcado é um sistema computacional microprocessado com uma função dedicada dentro de um sistema maior. Diferente de um PC, ele é projetado para tarefas específicas (ex: controle de um micro-ondas ou freios ABS).

## 1.2 Hardware Open-Source e Movimento Maker

- **Hardware Livre:** Projetos cujos esquemas e arquivos de fabricação são públicos (ex: Arduino).
- **Tinkering:** A arte de "mexer", experimentar e prototipar rapidamente.

## 1.3 Internet das Coisas (IoT)

A integração de dispositivos embarcados com a internet, permitindo a coleta de dados e o controle remoto de dispositivos físicos.

---

# Módulo 2: Princípios de Elétrica e Eletrônica

## 2.1 Ferramentas de Prototipagem

- **Protoboard:** Placa de ensaio que permite montar circuitos sem solda.
- **Componentes Básicos:**
  - **LEDs:** Diodos que emitem luz (exigem resistores para não queimar).
  - **Botões (Push-buttons):** Chaves para entrada de sinal digital.
- **Módulos e Shields:** Placas prontas que expandem as funções do microcontrolador (ex: Shield Ethernet, Módulo Bluetooth).

---

# Módulo 3: Programação de Microcontroladores

## 3.1 Estrutura do Programa

Todo código em ambiente Arduino possui duas funções principais:

1. `setup()`: Executa uma única vez ao ligar o dispositivo. Usada para configurações.
2. `loop()`: Executa repetidamente em ciclo infinito.

| Tipo  | Descrição        | Exemplo            |
|-------|------------------|--------------------|
| int   | Inteiros         | int contador = 10; |
| float | Números decimais | float temp = 25.5; |

| Tipo | Descrição           | Exemplo             |
|------|---------------------|---------------------|
| bool | Verdadeiro ou Falso | bool estado = true; |
| char | Caractere único     | char letra = 'A';   |

### 3.3 Estruturas de Decisão e Repetição

- **if / else:** Tomada de decisão baseada em condições.
- **for / while:** Repetição de blocos de código enquanto uma condição for atendida.

---

## Módulo 4: Entrada e Saída (I/O)

### 4.1 Digital (0 ou 1)

- `pinMode(pino, modo):` Define se o pino é INPUT ou OUTPUT.
- `digitalWrite(pino, valor):` Envia sinal HIGH (5V/3.3V) ou LOW (0V).
- `digitalRead(pino):` Lê se o estado é alto ou baixo.

### 4.2 Analógica (Valores Variáveis)

- **ADC (Conversor Analógico-Digital):** Transforma uma voltagem (0-5V) em um número (0-1023).
- `analogRead(pino):` Lê sensores como potenciômetros ou LDRs.
- `analogWrite(pino, valor):` Simula saída analógica via **PWM** (útil para brilho de LEDs e velocidade de motores).

---

## Módulo 5: Controle de Tempo e Funções Avançadas

### 5.1 Funções de Delay vs. Millis

- `delay():` Pausa o programa completamente (bloqueante).
- `millis():` Retorna o tempo decorrido desde o início do programa. Permite multitarefa (não bloqueante).

### 5.2 Bibliotecas e Comunicação

As bibliotecas facilitam o uso de periféricos complexos:

- **Sensores:** Bibliotecas para DHT11 (umidade/temperatura).
  - **Comunicação:** Protocolos como **I2C**, **SPI** e **Serial (UART)**.
-

## **Exercício Prático Sugerido**

### **Projeto: Semáforo Inteligente com Botão de Pedestre**

1. Configure 3 LEDs (Verde, Amarelo, Vermelho).
  2. Use um botão para solicitar a travessia.
  3. Implemente a lógica de tempo usando as funções de controle de tempo aprendidas.
-